

Técnicas de Projeto de Algoritmos

Yuri Kaszubowski Lopes

UDESC

Conteúdo

- Quicksort: divisão e conquista (D&C)
- Fibonacci: força bruta (FB), programação dinâmica (PD, DP)
- Moeda: FB, PD e Guloso (Greedy)

Quicksort

- **Divisão:** Particionar (reorganizar) o arranjo $A[p \dots r]$ em dois subarranjos (possivelmente vazios) $A[p \dots q - 1]$ e $A[q + 1 \dots r]$ tais que:
 - ▶ cada elemento de $A[p \dots q - 1]$ é menor ou igual a $A[q]$,
 - ▶ cada elemento de $A[q + 1 \dots r]$ é maior a $A[q]$.
 - ▶ Calcular o índice q como parte desse procedimento de particionamento
- **Conquista:** Ordenar os dois subarranjos $A[p \dots q - 1]$ e $A[q + 1 \dots r]$ por chamadas recursivas a quicksort.
- **Combinação:** Como os subarranjos já estão ordenados, não é necessário nenhum trabalho para combiná-los: o arranjo $A[p \dots r]$ inteiro agora está ordenado.

Quicksort

1	8	7	5	6	4	2	3	9	10	12	11
0	1	2	3	4	5	6	7	8	9	10	11

Quicksort

1	2	3	5	6	4	8	7	9	10	12	11
0	1	2	3	4	5	6	7	8	9	10	11

Quicksort

1	2	3	5	6	4	7	8	9	10	12	11
0	1	2	3	4	5	6	7	8	9	10	11

Quicksort

```
1 void swap(int *a, int *b) {
2     int aux = *a;
3     *a = *b;
4     *b = aux;
5 }
6
7 int partition(int v[], int p, int r) {
8     int pivot = v[r];
9     int i = p;
10
11     for (int j = p; j < r; j++) {
12         if (v[j] < pivot) {
13             swap(&v[i], &v[j]);
14             i++;
15         }
16     }
17     swap(&v[i], &v[r]);
18     return i;
19 }
```

Quicksort

```
1 void quicksort(int v[], int lenght, int p, int r) {
2     if (p >= r)
3         return;
4
5     int q = partition(v, p, r);
6     quicksort(v, lenght, p, q-1);
7     quicksort(v, lenght, q+1, r);
8 }
9
10 void qs(int v[], int lenght) {
11     quicksort(v, lenght, 0, lenght-1);
12 }
```

Quicksort

Pior Caso:

$$T(n) = T(n-1) + T(0) + \Theta(n)$$

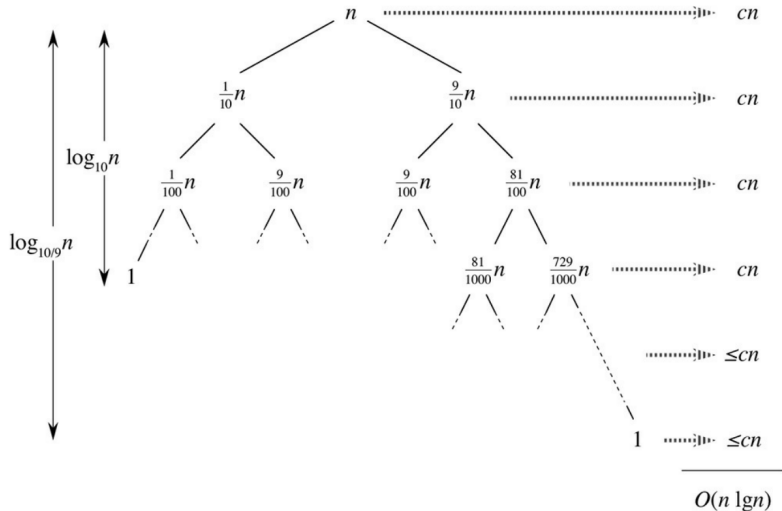
Melhor Caso:

$$T(n) = 2T((n-1)/2) + \Theta(n)$$

Intuição para o caso médio

- Qual das possíveis permutações é a entrada? O que é comum?
 - ▶ $T(n) = T(4n/10) + T(6n/10) + \Theta(n)$
 - ▶ $T(n) = T(9n/10) + T(1n/10) + \Theta(n)$ (particionamento 9 para 1)
 - ▶ Para $T(9n/10)$, temos alturas $\log_{10/9} n$ e $\log_{10} n$
 - ▶ Até nível $\log_{10} n$, todo nível tem custo cn
 - ▶ Após nível $\log_{10} n$, todo nível tem custo $\leq cn$
 - ▶ O custo é $O(n \lg n)$
 - ▶ Até mesmo para particionamento 99 para 1
- Improvável que ocorra da mesma forma em todos os níveis: mix de repartições equilibradas e desequilibradas
- Boas partições compensam má partições (grande maioria dos casos)

Quicksort



Fibonacci

Definição Fibonacci

$$F_n = F_{n-1} + F_{n-2}$$

Exemplo: 0, 1, 1, 2, 3, 5, 8, 13, ...

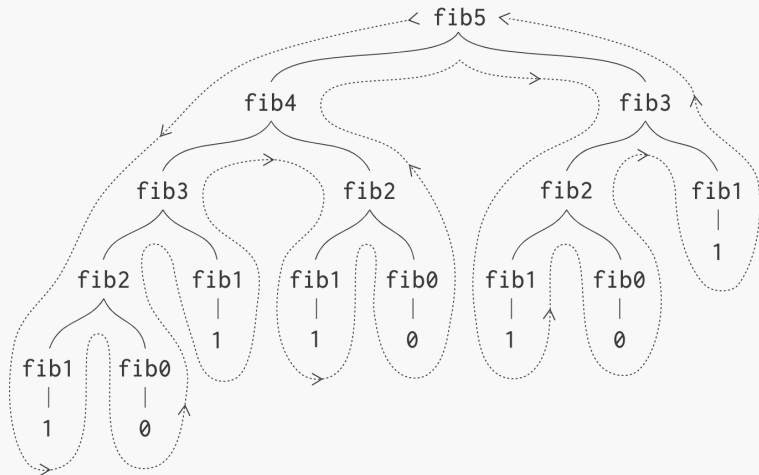
Força Bruta:

```
1 def fibo(n):  
2     if n == 0:  
3         return 0  
4     if n == 1:  
5         return 1  
6     else:  
7         return fibo(n-1) + fibo(n-2)  
8
```

- `fibo(8)`: 21 (24 ms)
- `fibo(10)`: 55 (25 ms)
- `fibo(20)`: 6765 (30 ms)
- `fibo(30)`: 832040 (309 ms)
- `fibo(40)`: 102334155 (38 s 553 ms)

Fibonacci

Problema



Fibonacci: Programação Dinâmica

Solução PD (DP)

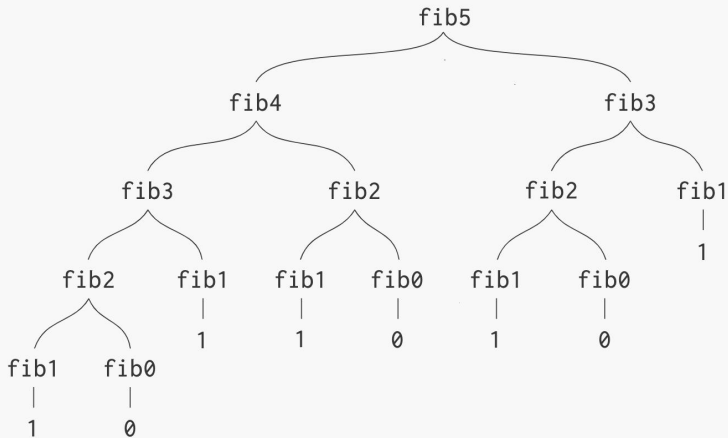
- Manter uma tabela com soluções já calculadas

```
1 mem = {  
2     0: 0,  
3     1: 1,  
4 }  
5 def fibo3(n):  
6     if n not in mem:  
7         mem[n] = fibo3(n-1) + fibo3(n-2)  
8     return mem[n]  
9
```

- `fibo3(8)`: 21 (25 ms)
- `fibo3(10)`: 55 (26 ms)
- `fibo3(40)`: 102334155 (27 ms)
- `fibo3(100)`: 354224848179261915075 (23 ms)
- `fibo3(999)`: NUMERO GIGANTE (28 ms)

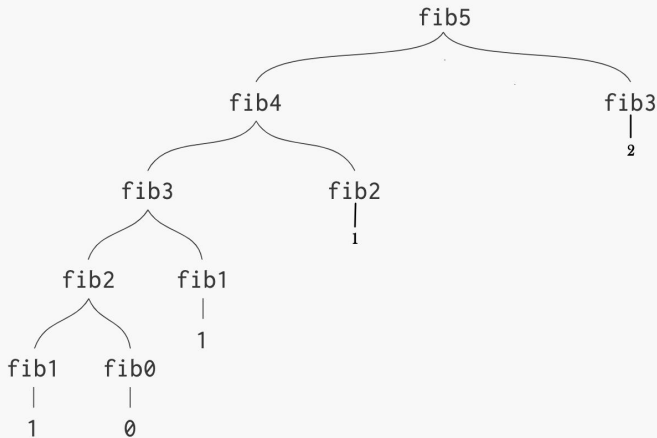
Fibonacci: Programação Dinâmica

Problema



Fibonacci: Programação Dinâmica

Problema



Moeda

- Qual a menor quantidade de moedas para formar um valor?
- diferentes denominações, diferentes estratégias:
 - ▶ Moedas de 100, 50, 25, 10, 5, 1 (canonico)
 - ▶ Moedas de 6, 4, 1 (não canonico)
 - ★ Para 8: (4+4: ótimo), (6+1+1: guloso)
 - ▶ $\frac{1}{2}$, 1, 3, 6, 12 (shiling), 24 (florin), 30 (half-crown), 60 (crown) e 240 (pound):
 - ★ System Inglês usado antes de 1971: exemplo de sistema naturalmente não canônico
 - ★ 48 pence (24+24: ótimo), (30+12+6: guloso)
- Estratégia: A cada chamada recursiva selecionar uma moeda $M \leq valor$
- Chamar recursivamente a função com $valor' = valor - M$

Moeda: Força bruta

- Qual moeda selecionar?
 - ▶ Em FB teste todas, selecione a com o melhor resultado.
- Resposta ótima, mas demorado

```
1 int moeda_FB(int valor, int* moedas, int n) {
2     int res, min = 9999999;
3
4     if (valor == 0)
5         return 0;
6
7     for (int i=0; i < n; i++) {
8         if (moedas[i] <= valor) {
9             res = moeda_FB(valor - moedas[i], moedas, n);
10            if (res < min)
11                min = res;
12        }
13    }
14    return min + 1;
15 }
```

Moeda: Guloso

- Qual moeda selecionar?
 - ▶ A maior moeda possível (mas com denominação *leq* que o valor restante)
- Rápido, mas não generalista

```
1 int moeda_GULOSO(int valor, int* moedas, int n) {
2     int res;
3
4     if (valor == 0)
5         return 0;
6
7     for (int i=0; i < n; i++) {
8         if (moedas[i] <= valor) {
9             res = moeda_GULOSO(valor - moedas[i], moedas, n);
10            break;
11        }
12    }
13
14    return res + 1;
15 }
```

Moeda: PD

- Qual moeda seleccionar?

- ▶ Como em FB teste todas, mas se já resolveu para este valor use a solução armazenada

```
1 int dp[MAX_VAL];
2
3 int init() {
4     dp[0] = 0;
5     for (int i = 1; i < MAX_VAL; i++)
6         dp[i] = -1;
7 }
8
9 int moeda_PD(int valor, int* moedas, int n) {
10     int res;
11
12     if (dp[valor] == -1) {
13         dp[valor] = 9999999;
14         for (int i=0; i < n; i++) {
15             if (moedas[i] <= valor) {
16                 res = moeda_PD(valor - moedas[i], moedas, n) + 1;
17                 if (res < dp[valor])
18                     dp[valor] = res;
19             }
20         }
21 }
```

Moeda: Tempos

```
1 int V[N] = {6, 4, 1};  
2  
3 res = moeda_FB(50, V, N); // 9 em 1.779s  
4  
5 res = moeda_GULOSO(50, V, N); // 10 em 0.002s  
6  
7 res = moeda_PD(50, V, N); // 9 em 0.002s
```

Técnicas de Projeto de Algoritmos

Yuri Kaszubowski Lopes

UDESC